

IIC2413 Base de Datos - Interrogación 1 by Franco Nicolai

Clase 0: ¿Qué es una Base de Datos?

[Conceptos Clave](#)

[Tablas de Ejemplo para explicar contenidos](#)

Clase 1: Modelo Relacional

[Conceptos Clave del Modelo Relacional](#)

[Esquema e Instancia](#)

[Restricciones de Integridad](#)

Clase 2: Álgebra Relacional

[¿Qué es el Álgebra Relacional?](#)

[Operadores Fundamentales](#)

[Propiedades de las Consultas](#)

Clase 3: Diseño de Bases de Datos (**Análisis de Requisitos → Diseño Conceptual (Modelo Entidad-Relación)**)

[El Proceso de Diseño de una Base de Datos](#)

[Profundizando en el Modelo Entidad-Relación](#)

Clase 4: Diseño de Bases de Datos (de **Diseño Conceptual (Modelo Entidad-Relación) → Diseño Lógico (Modelo Relacional)**)

[Etapas de transformación](#)

[Restricciones de Integridad](#)

Clase 5: Dependencias y Formas Normales

[Dependencias Funcionales](#)

[Formas Normales](#)

Clase 6 y 7: SQL - Structured Query Language

[Tipos de Datos Comunes](#)

[DDL: Definición de la Estructura](#)

[DML: Manipulación de los Datos](#)

[Operaciones con Múltiples Tablas](#)

[Funciones de Agregación](#)

[Consultas Anidadas y Operadores de Conjunto](#)

[Manejo de Valores Nulos \(NULL \)](#)

Clase 0: ¿Qué es una Base de Datos?

Una **base de datos (BD)** es como una gran biblioteca digital donde se guarda toda esta información de forma organizada y estructurada. Esto permite que la información sea fácil de encontrar, actualizar y analizar. En lugar de tener la información desordenada en miles de archivos de texto, la BD la organiza en tablas.

Conceptos Clave

- **Dato:** Un valor único e individual. Por ejemplo, "Juan Pérez", "5.000.000" o "25-09-2025".
- **Información:** El resultado de procesar o contextualizar los datos. Por ejemplo, "El cliente Juan Pérez tiene un saldo de 5.000.000 pesos". La información es lo que le da sentido a los datos.
- **Sistema de Gestión de Bases de Datos (SGBD):** El software que nos permite gestionar la base de datos. Es como el bibliotecario de nuestra biblioteca digital. Nos ayuda a crear, leer, actualizar y eliminar datos (esto se conoce como operaciones **CRUD**, del inglés Create, Read, Update, Delete). Un ejemplo de SGBD en el mundo real podría ser PostgreSQL, Oracle o MySQL.

Tablas de Ejemplo para explicar contenidos

Para que sea más fácil seguir los contenidos, usaremos las siguientes tres tablas que representan la información de un banco:

CLIENTES

ID_Cliente	Nombre	Saldo
C001	Juan Pérez	\$5,000,000
C002	María López	\$12,500,000
C003	Pedro Gómez	\$750,000

TRANSACCIONES

ID_Transaccion	Nro_Cuenta	Monto	Fecha
T001	1001	\$500,000	2024-05-15
T002	1003	\$1,200,000	2024-05-16
T003	1002	\$100,000	2024-05-16

CUENTAS

Nro_Cuenta	ID_Cliente	Tipo	Saldo
1001	C001	Ahorro	\$2,000,000
1002	C001	Corriente	\$3,000,000
1003	C002	Ahorro	\$12,500,000
1004	C003	Corriente	\$750,000

Clase 1: Modelo Relacional

El **modelo relacional**, creado por Edgar F. Codd, es una forma de organizar los datos en bases de datos. Su nombre viene de la idea de que los datos se organizan en "relaciones" (que nosotros llamamos tablas).

Conceptos Clave del Modelo Relacional

- **Relación:** En términos simples, es una tabla.

- **Atributo:** Es una columna en la tabla.
- **Tupla:** Es una fila en la tabla.
- **Dominio:** El tipo de dato que puede tener un atributo. Por ejemplo, el atributo `Saldo` podría tener un dominio de números enteros o decimales.

Ejemplo: En nuestro banco, podríamos tener una relación llamada `CLIENTES` con los siguientes atributos: `ID_Cliente`, `Nombre` y `Saldo`. Cada fila en esta tabla representaría a un cliente individual.

Esquema e Instancia

- **Esquema:** Es un conjunto de relaciones con sus atributos. Un esquema define el nombre de las relaciones y sus atributos.
 - **Ejemplo de esquema del banco:** `CLIENTES(ID_Cliente, Nombre, Saldo)`
- **Instancia:** Son los datos reales en un momento dado que siguen la estructura del esquema. Una instancia es una "foto" de la base de datos en un momento particular.
 - **Ejemplo de instancia del banco:** La tabla con los datos reales de Juan, María y Pedro que estan registrados.

Restricciones de Integridad

Las **restricciones de integridad** son reglas que aseguran que los datos en la base de datos sean correctos y consistentes. La restricción más importante son las llaves.

- **Llave:** Es un atributo (o conjunto de atributos, o sea, pueden ser compuestas) que identifica de manera única cada tupla en una relación.
 - **Ejemplo:** En la tabla `CLIENTES`, `ID_Cliente` es una llave, ya que no puede haber dos clientes con el mismo ID.
- **Super llave:** Cualquier conjunto de atributos que determina a todos los demás atributos en una relación. Una super llave puede contener atributos extra, incluso si no son necesarios para la identificación única.
 - **Ejemplo:** En la tabla `CLIENTES`, tanto `{ID_Cliente}` como `{ID_Cliente, Nombre}` serían super llaves, ya que ambas identifican de manera única a una tupla. Sin embargo, `{ID_Cliente, Nombre}` tiene un atributo extra (`Nombre`).
- **Llaves candidatas:** Una llave candidata es un conjunto mínimo de atributos que puede ser una súper llave. Es decir, que no tiene atributos extra.
 - **Ejemplo:** En nuestra tabla `CLIENTES`, si además del `ID_Cliente` tuviéramos un atributo `RUT`, tanto `{ID_Cliente}` como `{RUT}` serían llaves candidatas, ya que por sí solas identifican a cada cliente y no contienen atributos extra. Sin embargo, `{ID_Cliente, Nombre}` no sería una llave candidata, ya que `{ID_Cliente}` ya es una llave por sí sola.
- **Llave Primaria:** Una de las llaves candidatas que elegimos para ser la llave principal de la tabla. Generalmente la subrayamos en el esquema.
 - **Ejemplo:** `CLIENTES(ID_Cliente, Nombre, Saldo)`
- **Llave Subrogada (Surrogate Key):** Una llave genérica, generalmente artificial, que se usa para simplificar las cosas. Se utiliza cuando no existe un atributo natural que pueda servir como llave primaria. Por ejemplo, en lugar de usar una combinación de nombre y fecha de nacimiento, es mucho más simple usar un `ID_Cliente` autoincrementable.

Clase 2: Álgebra Relacional

En la clase 1, aprendimos a organizar los datos. Ahora, en la clase 2, veremos cómo interactuar con ellos usando un lenguaje teórico, el **Álgebra Relacional**.

¿Qué es el Álgebra Relacional?

El Álgebra Relacional es un lenguaje formal para manipular datos en el modelo relacional. Se basa en una serie de operadores que, como su nombre lo indica, operan sobre relaciones (tablas) y devuelven nuevas relaciones. Esto es fundamental, ya que los lenguajes de consulta no modifican las tablas originales, sino que generan nuevas tablas con la información solicitada. Para explicar de forma más sencilla, seguiremos usando las tablas explicadas anteriormente, en particular las tablas `CLIENTES` y `CUENTAS` de nuestro banco:

Tabla CLIENTES

ID_Cliente	Nombre	Saldo
C001	Juan Pérez	\$5,000,000
C002	María López	\$12,500,000
C003	Pedro Gómez	\$750,000
C04	N/A	N/A

Tabla CUENTAS

Nro_Cuenta	ID_Cliente	Tipo	Saldo
1001	C001	Ahorro	\$2,000,000
1002	C001	Corriente	\$3,000,000
1003	C002	Ahorro	\$12,500,000
1004	C003	Corriente	\$750,000

Operadores Fundamentales

Existen varios operadores:

- **Selección (σ):** Este operador filtra las filas de una relación. Es como decir: "dame todas las tuplas que cumplan con una condición específica". Teóricamente se denota como:
 $\sigma_{condición}(R)$ donde `R` es la relación (tabla) y `condición` es una expresión que evalúa a verdadero o falso, usando operadores como: `=`, `!=`, `<`, `≤`, `>`, `≥`. Estas condiciones se pueden combinar usando `AND` (`∧`), `OR` (`∨`) y `NOT` (`¬`).

Ejemplo: Si en nuestra tabla `CLIENTES`, queremos encontrar a todos los clientes que tienen un saldo mayor a 1000, la operación se vería así:

$\sigma_{(Saldo>1000)\vee(Nombre="Juan")}(CLIENTES)$

Resultado:

ID_Cliente	Nombre	Saldo
C001	Juan Pérez	\$5,000,000
C002	María López	\$12,500,000

- **Proyección (π):** Este operador selecciona las columnas de una relación. Con él, podemos elegir qué atributos (columnas) queremos ver. Su notación teórica es:
 $\pi_{A_1,A_2,...,A_n}(R)$ donde `A` son los atributos que se desean y `R` es la relación (tabla).

Ejemplo: Si solo nos interesa el nombre y el saldo de nuestros clientes, la operación sería:

Resultado:

$\pi_{Nombre,Saldo}$ (CLIENTES)

Nombre	Saldo
Juan Pérez	\$5,000,000
María López	\$12,500,000
Pedro Gómez	\$750,000

- **Unión (U):** Este operador combina dos relaciones. Para que la unión funcione, ambas relaciones deben tener el mismo número y tipo de atributos. Observación: Las relaciones son conjuntos de tuplas, por lo que si una tupla está en ambas, la unión la contiene una sola vez.

Ejemplo: Si tenemos una tabla CLIENTES_VIP con los mismos atributos que CLIENTES, podemos unir las para tener una sola lista de todos los clientes.

CLIENTES U NUEVOS_CLIENTES

Tabla NUEVOS_CLIENTES:

ID_Cliente	Nombre	Saldo
C004	Ana Torres	\$1,500,000
C001	Juan Pérez	\$5,000,000

Resultado:

ID_Cliente	Nombre	Saldo
C001	Juan Pérez	\$5,000,000
C002	María López	\$12,500,000
C003	Pedro Gómez	\$750,000
C004	Ana Torres	\$1,500,000

- **Renombrado (ρ):** Este operador sirve para cambiar el nombre de una relación o de sus atributos. Es muy útil cuando realizamos operaciones complejas y queremos dar un nombre a la nueva tabla resultante para poder referirnos a ella más adelante. Su notación es: $\rho_{nuevo_nombre}(R)$.

- **Ejemplo:** Si queremos cambiar el nombre de la tabla CLIENTES a CLIENTELA y el atributo Nombre a Nombre_Completo, la operación sería:

$\rho_{CLIENTELA(ID_Cliente,Nombre_Completo,Saldo)}$ (CLIENTES)

- **Producto Cruz (×):** Este operador, también conocido como producto cartesiano, nos permite combinar información de dos tablas que no necesariamente tienen atributos en común. El resultado es una nueva tabla que tiene el total de filas de ambas tablas multiplicadas. Cada tupla (fila) de la primera tabla se combina con cada tupla (fila) de la segunda tabla.

Ejemplo: Si queremos combinar la información de la tabla CLIENTES con la tabla CUENTAS para generar una lista de todas las combinaciones posibles, la operación sería:

CLIENTES × CUENTAS

Resultado:

ID_Cliente	Nombre	Saldo	ID_Cuenta	ID_Cliente (Cuentas)	Tipo.
C001	Juan Pérez	\$5,000,000	1001	C001	Ahorro
C001	Juan Pérez	\$5,000,000	1002	C001	Corriente
C001	Juan Pérez	\$5,000,000	1003	C002	Ahorro
C001	Juan Pérez	\$5,000,000	1004	C003	Corriente
C002	María López	\$12,500,000	1001	C001	Ahorro
C002	María López	\$12,500,000	1002	C001	Corriente
C002	María López	\$12,500,000	1003	C002	Ahorro
C002	María López	\$12,500,000	1004	C003	Corriente
C003	Pedro Gómez	\$750,000	1001	C001	Ahorro
C003	Pedro Gómez	\$750,000	1002	C001	Corriente
C003	Pedro Gómez	\$750,000	1003	C002	Ahorro
C003	Pedro Gómez	\$750,000	1004	C003	Corriente

- **Join (⋈):** Este operador es una de las operaciones más importantes y comunes en el álgebra relacional. Permite combinar tuplas de dos relaciones que tienen un atributo en común, a diferencia del Producto Cruz que combina todas las tuplas. El Join es conceptualmente equivalente a un **Producto Cruz** seguido de una **Selección**. **Equivalencia:** $R_1 \bowtie_{condición} R_2 = \sigma_{condición} (R_1 \times R_2)$.

Ejemplo: Si queremos obtener una lista de clientes y la información de sus cuentas, podemos hacer un *join* de las tablas CLIENTES y CUENTAS sobre el atributo ID_Cliente. La operación se vería así:

$\bowtie_{CLIENTES.ID_Cliente=CUENTAS.ID_Cliente}$

CUENTAS

Resultado:

ID_Cliente	Nombre	Saldo (Clientes)	Nro_Cuenta	ID_Cliente (Cuentas)	Tipo	Saldo (Cuentas)
C001	Juan Pérez	\$5,000,000	1001	C001	Ahorro	\$2,000,000
C001	Juan Pérez	\$5,000,000	1002	C001	Corriente	\$3,000,000
C002	María López	\$12,500,000	1003	C002	Ahorro	\$12,500,000
C003	Pedro Gómez	\$750,000	1004	C003	Corriente	\$750,000

- Este resultado es mucho más útil que el producto cruz, ya que solo muestra las filas que tienen una relación lógica entre sí, es decir, solo se combinan los clientes con sus propias cuentas.

- **Intersección (∩):** Este operador es una operación de conjuntos. Devuelve todas las tuplas que existen en ambas relaciones. Para que la intersección sea válida, las dos relaciones deben tener exactamente los mismos atributos, tanto en cantidad como en tipo.

Ejemplo: Si tenemos una tabla de CLIENTES_PREMIUM con los mismos atributos que la tabla CLIENTES, podemos encontrar los clientes que son parte de ambos grupos.

Tabla CLIENTES_PREMIUM:

ID_Cliente	Nombre	Saldo
C001	Juan Pérez	\$5,000,000
C005	Luis Ramos	\$25,000,000

Resultado:

ID_Cliente	Nombre	Saldo
C001	Juan Pérez	\$5,000,000

- **Diferencia (–):** Este operador también es una operación de conjuntos. Devuelve las tuplas que están en la primera relación, pero no en la segunda. Similar a la intersección, ambas relaciones deben tener los mismos atributos.

Ejemplo: Usando las mismas tablas anteriores, la operación `CLIENTES - CLIENTES_PREMIUM`.

Resultado:

ID_Cliente	Nombre	Saldo
C002	María López	\$12,500,000
C003	Pedro Gómez	\$750,000

Propiedades de las Consultas

Una propiedad importante que tienen muchos operadores del álgebra relacional es la **monotonía**.

- **Teorema de Monotonía:** Toda consulta que utiliza solo los operadores de selección (σ), proyección (π), unión ($\cup$), producto cruz ($\times$), y join ($\Join$) es **monótona**.
- **¿Qué significa esto?:** En términos sencillos, una consulta es monótona si el resultado de la consulta **nunca se hace más pequeño** al añadir más datos a la base de datos. Si agregas una nueva tupla a una tabla, la nueva tupla puede aparecer en el resultado de la consulta (o no), pero nunca hará que una tupla que ya estaba en el resultado desaparezca. Esto no se cumple para los operadores de intersección y diferencia.

Clase 3: Diseño de Bases de Datos (Análisis de Requisitos → Diseño Conceptual (Modelo Entidad-Relación))

En las clases anteriores, nos enfocamos en el **Modelo Relacional** y en cómo usar el **Álgebra Relacional** para realizar consultas. Con esos conocimientos, ya podríamos empezar a crear una base de datos y a interactuar con ella. Sin embargo, surge una pregunta clave: **¿Sabemos si la estamos diseñando bien?**

El Proceso de Diseño de una Base de Datos

El diseño de una base de datos es un proceso estructurado que se puede dividir en tres etapas principales.

1. **Análisis de Requisitos:** En esta etapa, nos comunicamos con los usuarios para entender exactamente lo que necesitan. El objetivo es identificar las entidades claves, los datos que se deben almacenar, y las relaciones entre ellos. Es un proceso de recopilación de información que nos ayudará a definir un buen modelo conceptual.
2. **Diseño Conceptual (Modelo Entidad-Relación):** Una vez que hemos recopilado los requisitos, los convertimos en un **modelo conceptual**. Para esto, utilizamos una herramienta llamada **Modelo Entidad-Relación (E-R)**. Este modelo nos permite representar de forma abstracta y gráfica el mundo real.
3. **Diseño Lógico (Modelo Relacional):** En la etapa final, convertimos el **Modelo Entidad-Relación** a un **esquema relacional** concreto. Aquí, las entidades se transforman en tablas, los atributos en columnas y las relaciones en **llaves foráneas** que vinculan las tablas. Este es el punto en el que el diseño abstracto se convierte en la estructura de tablas que ya conocemos.

Profundizando en el Modelo Entidad-Relación

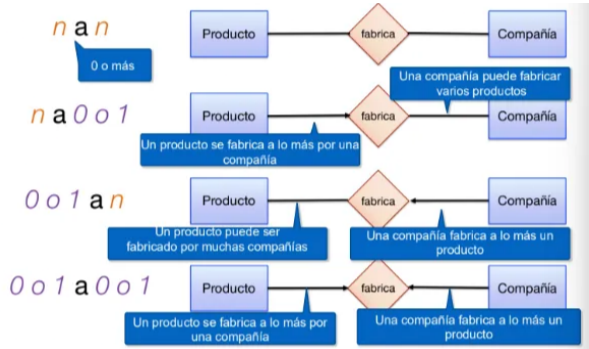
El **Modelo Entidad-Relación (E-R)** es la base de un buen diseño. Para entenderlo, es fundamental conocer sus componentes principales:

- **Entidad:** Una entidad representa un conjunto de objetos que son similares y que nos interesa almacenar. Por ejemplo, en nuestro banco, `CLIENTES` y `CUENTAS` son entidades.
- **Entidad débil:** Una entidad débil es aquella que no puede ser identificada de forma única solo con sus propios atributos. Su llave depende de la llave de otra entidad, llamada **dueño identificador** (en inglés, *identifying owner*). La llave de la entidad débil se conoce como **llave parcial**.
 - **Ejemplo:** Consideremos una entidad llamada `DEPENDIENTES` que representa a los familiares de los empleados de una empresa. Un dependiente no puede ser identificado únicamente por su nombre, ya que podría haber dos dependientes con el mismo nombre. Para identificarlo de forma única, necesitamos saber a qué empleado pertenece. Por lo tanto, `DEPENDIENTES` es una **entidad débil**, y `EMPLEADOS` es su **dueño identificador**. La **llave parcial** de la entidad `DEPENDIENTES` es el nombre, ya que un dependiente solo puede ser identificado de forma única en relación con su empleado.
- **Atributo:** Son las propiedades o cualidades que describen a una entidad. Cada atributo pertenece a un **dominio**, que define el conjunto de valores posibles que puede tomar.

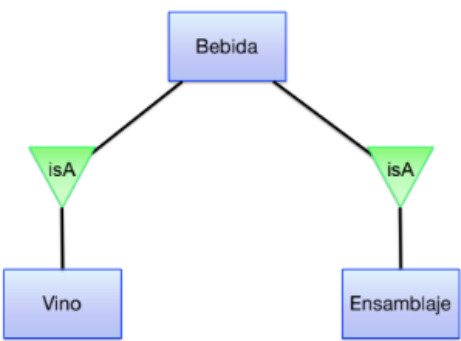


- **Relación:** Una relación es una asociación entre dos o más entidades (usualmente solo asocian a dos entidades). Nos permite modelar cómo se conectan los objetos del mundo real. Por ejemplo, en el banco, existe una relación entre la entidad `CLIENTES` y la entidad `CUENTAS` (un cliente es dueño de una o más cuentas). Las relaciones también pueden tener atributos descriptivos que dan información de la relación.





- **Jerarquía:** Se utiliza para modelar entidades que tienen características comunes y otras más específicas. Un ejemplo podría ser una entidad **PERSONA** que puede ser de tipo **CLIENTE** o **EMPLEADO**. La jerarquía es un tipo de relación "Is A". **Solapamiento de herencia:** Puede o no haber solapamiento en la herencia, lo que significa que un mismo objeto del mundo real puede pertenecer a más de una de las entidades de la jerarquía. Por ejemplo, un empleado de nuestro banco puede ser a la vez un cliente.



Obligatorio: Cada entidad debe tener un atributo llave (identificador) que la hace única. En un diagrama E-R, este atributo se subraya con una línea continua. Sin embargo, una entidad débil tiene una **llave parcial** (identificador parcial) que, en un diagrama, se representa con una línea discontinua.

Clase 4: Diseño de Bases de Datos (de Diseño Conceptual (Modelo Entidad-Relación) → Diseño Lógico (Modelo Relacional))

En la clase anterior, comenzamos el proceso de diseño de bases de datos, que va desde los **requisitos de usuario** hasta el **modelo relacional**. Hoy profundizaremos en la segunda etapa: cómo transformar un **Modelo Entidad-Relación (E-R)** en un **modelo relacional**. Para ello el proceso es analizar etapa a etapa:

Etapas de transformación

1. Transformación de Entidades

Cada **entidad** en nuestro modelo E-R se convierte directamente en una **tabla** en el modelo relacional. Los **atributos** de la entidad se transforman en las **columnas** de la tabla.

- **Regla:** Entidad → Tabla
- **Regla:** Atributo → Columna

Por ejemplo, si tenemos la entidad **CLIENTES** con los atributos **ID_Cliente**, **Nombre** y **Saldo**, en el modelo relacional se convierte en la tabla **CLIENTES** con las columnas **ID_Cliente**, **Nombre** y **Saldo**.

2. Llaves (repaso de contenidos anteriores)

Las **llaves** son fundamentales en el modelo relacional para identificar de manera única cada fila (tupla) de una tabla.

- **Llave Primaria (Primary Key):** Identifica de forma única cada tupla en una tabla. Por ejemplo, **ID_Cliente** es la llave primaria de la tabla **CLIENTES**. Es crucial que este atributo no se repita y no tenga valores nulos.
- **Llave Foránea (Foreign Key):** Es un atributo que establece una relación al hacer referencia a la llave primaria de otra tabla. A diferencia de las llaves primarias, una llave foránea puede tener valores repetidos y, en algunos casos, nulos. Por ejemplo, en la tabla **CUENTAS**, el atributo **ID_Cliente** es una llave foránea que apunta al **ID_Cliente** de la tabla **CLIENTES**, lo que vincula una cuenta con su dueño.

Cuando pasamos un diagrama Entidad-Relación a un modelo relacional, debemos indicar las llaves primarias y foráneas para establecer la estructura. Así, para las entidades **Producto**, **Compañía** y la relación **Fabrica**, el resultado en el modelo relacional sería:

- **Producto**(nombre : string, precio : int, categoría : string)
- **Compañía**(nombre : string, valor-acción : int)
- **Fabrica**(Producto.nombre : string, Compañía.nombre : string, desde : date)

Aquí, **Producto.nombre** y **Compañía.nombre** en la tabla **Fabrica** son llaves foráneas que se refieren a las llaves primarias de las tablas **Producto** y **Compañía**, respectivamente.

3. Relaciones

El tercer paso es manejar las **relaciones** entre entidades. La forma de transformar una relación a tablas depende de su **cardinalidad** (cómo se relacionan las tuplas de una tabla con las de otra).

- **Relación 1:1**
 - **Descripción:** Una tupla de la entidad A se relaciona con una y solo una tupla de la entidad B, y viceversa.
 - **Ejemplo:** Un cliente puede tener una única cuenta principal, y cada cuenta principal le pertenece a un solo cliente.
 - **Transformación:** La llave primaria de una de las tablas se añade como llave foránea en la otra. No importa en cuál de las dos tablas se añada la llave foránea, ya que la relación es uno a uno.
- **Relación 1:N**

- **Descripción:** Una tupla de la entidad A puede relacionarse con muchas tuplas de la entidad B, pero una tupla de B se relaciona con solo una de A.
- **Ejemplo:** Un cliente puede tener varias cuentas de ahorro, pero cada cuenta de ahorro le pertenece a un solo cliente.
- **Transformación:** La llave primaria de la tabla "uno" (la entidad A) se añade como llave foránea en la tabla "muchos" (la entidad B). En este caso, el `ID_Cliente` de la tabla `CLIENTES` se convierte en una llave foránea en la tabla `CUENTAS`.

- **Relación N:M**

- **Descripción:** Una tupla de la entidad A puede relacionarse con muchas tuplas de la entidad B, y una tupla de B puede relacionarse con muchas de A.
- **Ejemplo:** Un cliente puede tener varias propiedades, y una propiedad puede tener varios dueños.
- **Transformación:** Se crea una **nueva tabla** para la relación. Esta nueva tabla tendrá como llaves foráneas las llaves primarias de las dos tablas que se relacionan.

4. Jerarquía

Una **jerarquía** en el Modelo E-R, a menudo representada como un tipo de herencia "Es Un/a" (*Is A*), modela entidades que tienen atributos comunes y otros específicos.

Opciones de transformación a modelo relacional:

- **Opción 1:** Tablas solo para las subclases: En esta opción, se crea una tabla por cada subclase (la entidad hija). Cada tabla incluye los atributos de la superclase (la entidad padre) y sus propios atributos específicos.
 - **Ejemplo con una entidad `Bebida` y subclases `Vino` y `Cerveza`:**
 - `Vino(nombre : string, origen : string, año : int)`
 - `Cerveza(nombre : string, origen : string)`
- **Opción 2:** Una tabla para la superclase: Aquí, se crea una tabla para la superclase que contiene todos los atributos comunes. Adicionalmente, se crea una tabla para cada subclase que contiene solo sus atributos específicos, más una llave foránea que referencia a la llave primaria de la tabla de la superclase.
 - **Ejemplo:**
 - `Bebida(nombre : string, origen : string)`
 - `Vino(nombre : string, año : int)`
 - `Cerveza(nombre : string)`
- **Opción 3:** Quitar la jerarquía: Esta opción consiste en no usar jerarquía y crear una sola tabla para la superclase, incluyendo todos los atributos de las subclases.
 - **Ejemplo:**
 - `Bebida(nombre : string, origen : string, año : int, tipo : string)`
 - **Consideraciones:**
 - **Repetición de la columna `tipo`:** Es una columna redundante.
 - **Valores nulos (`nulls`):** Atributos como `año` (que solo aplica a `Vino`) tendrían valores nulos para las cervezas.
 - **Sencillez y compresión:** Es más simple de implementar, pero puede ser ineficiente si hay muchos atributos específicos y la mayoría de los valores son nulos.

¿Cuál es mejor?

La elección depende de las características de los datos:

- **Si hay mucho solapamiento entre las subclases**, la **Opción 2** es preferible. Si un mismo objeto puede ser de más de un tipo (ej: un empleado es también un cliente), una sola tabla para la superclase reduce la redundancia de datos.
- **Si no hay cobertura total** (es decir, si existen objetos de la superclase que no pertenecen a ninguna de las subclases), la **Opción 2** es la única viable. Por ejemplo, si hay otras bebidas aparte del vino y la cerveza (como el whisky) que no tienen su propia subclase, la tabla de la superclase (`Bebida`) puede guardarlas.
- **Si las consultas son por nombre de la superclase** (ej: buscar todas las `Bebida` s por nombre), la **Opción 2** es más eficiente, ya que solo se consulta una tabla en lugar de múltiples.

5. Entidades Débiles

Una entidad débil se convierte en una tabla. La **llave primaria** de esta nueva tabla es una **llave compuesta** por dos elementos:

1. La **llave parcial** de la entidad débil (el atributo que la identifica dentro del contexto de su dueño).
2. La **llave primaria** del dueño identificador, que se convierte en una llave foránea en la tabla de la entidad débil.

Ejemplo: La entidad `DEPENDIENTES` es débil y su dueño es la entidad `EMPLEADOS`.

- La **llave parcial** de `DEPENDIENTES` es `nombre`.
- La **llave primaria** de `EMPLEADOS` es `ID_Empleado`.

El modelo relacional para este caso sería:

- `EMPLEADOS(ID_Empleado : int, Nombre : string, Apellido : string)`
- `DEPENDIENTES(ID_Empleado : int, nombre : string, fecha_nacimiento : date)`

Aquí, el `ID_Empleado` en la tabla `DEPENDIENTES` es una **llave foránea** y, junto con el `nombre`, forman la **llave primaria** compuesta de la tabla `DEPENDIENTES`.

Restricciones de Integridad

Las restricciones de integridad son reglas que nos ayudan a mantener la consistencia y la exactitud de los datos.

- **Integridad de Entidad:** Asegura que cada tupla tenga un identificador único. Esto se logra con las llaves primarias, que no pueden ser nulas ni estar duplicadas.
- **Integridad Referencial:** Garantiza que las relaciones entre tablas sean válidas. Esto significa que un valor en una llave foránea debe existir como llave primaria en la tabla de referencia.
- **Integridad del Dominio:** Se encarga de que los valores de cada atributo estén dentro del rango permitido (por ejemplo, que el `Saldo` sea un número y no un texto).
- **Integridad de Clave:** La clave primaria de una tabla debe ser única para cada registro y no puede ser nula.

Con estos conceptos, ya tienes todas las herramientas necesarias para transformar un modelo conceptual en un diseño de base de datos sólido. ¿Te gustaría que profundicemos en algún tema en particular o pasemos a la próxima clase?

Clase 5: Dependencias y Formas Normales

Ahora que sabemos cómo transformar un modelo conceptual a un modelo relacional, surge una nueva pregunta: ¿cómo sabemos si nuestro diseño relacional es "bueno"? Un buen diseño debe evitar la **redundancia**, es decir, no guardar la misma información varias veces.

La redundancia en los datos puede causar problemas serios, conocidos como **anomalías**:

- **Anomalías de modificación:** Si un dato se repite, debes asegurarte de modificar todas sus copias. Si olvidas una, la base de datos se vuelve inconsistente.
- **Anomalías de inserción:** A veces no puedes insertar un dato si otro dato repetido o no relacionado aún no ha sido insertado.
- **Anomalías de eliminación:** Eliminar un dato puede borrar involuntariamente otra información que esté relacionada y que no quieres perder.

Para evitar estos problemas, usamos un conjunto de reglas llamadas **formas normales**. Antes de entenderlas, debemos comprender el concepto de **dependencia funcional**.

Dependencias Funcionales

Una **dependencia funcional** es una relación entre los atributos de una tabla. Decimos que un conjunto de atributos **X** determina funcionalmente a otro conjunto de atributos **Y** (escrito como $X \rightarrow Y$) si para cada tupla, el valor de **Y** está completamente determinado por el valor de **X**. Se denota como: $\pi_X(t1) = \pi_X(t2) \longrightarrow \pi_Y(t1) = \pi_Y(t2)$

- **Ejemplo:** Imagina una tabla para guías turísticos con los atributos (`gid`, `nombre`, `score`, `horas`, `valorHora`).
 - `gid` (ID del guía) es la llave primaria.
 - `score` es la calificación del guía.
 - `valorHora` es el pago por hora, que depende del `score`.

Aquí, podemos identificar varias dependencias funcionales:

- `gid` $\rightarrow$ `nombre`, `score`, `horas`, `valorHora`
 - Si conocemos el ID del guía, podemos determinar su nombre, su calificación, las horas que trabajó y su pago por hora. Esto es una dependencia obvia, ya que `gid` es la llave primaria.
- `score` $\rightarrow$ `valorHora`
 - El pago por hora solo depende de la calificación del guía. Si dos guías tienen el mismo `score`, tendrán el mismo `valorHora`.

Una **buena o mala dependencia** ocurre cuando una dependencia funcional es o no la **llave principal de la relación**. Una **buena** dependencia funcional es aquella donde el lado izquierdo de la dependencia es una superllave, asegurando que cada valor se asocie de forma única a un valor del lado derecho. Una **mala** dependencia funcional, en cambio, es aquella que provoca redundancia. Dos ejemplos clave de malas dependencias son las **dependencias parciales** (cuando un atributo depende de solo una parte de la llave compuesta) y las **dependencias transitivas** (cuando un atributo no-llave depende de otro atributo no-llave, en lugar de la llave principal). Estas malas dependencias son las que las formas normales buscan eliminar.

Ejemplo de una mala dependencia funcional:

Consideremos la tabla `Guías` con la siguiente información:

gid	nombre	score	horas	valorHora
101	Juan	5	10	5000
102	Ana	6	8	6000
103	Pedro	5	15	5000

En esta tabla, la dependencia funcional `score` $\rightarrow$ `valorHora` es una **mala dependencia** porque `score` no es una llave. Observa que el `valorHora` (5000) para un `score` de 5 se repite dos veces, lo que genera redundancia. Esta redundancia causa las anomalías mencionadas anteriormente.

Una **solución** para esta anomalía es **separar la tabla en tablas más pequeñas** de modo que cada una contenga una buena dependencia funcional.

Tabla de Guías:

gid	nombre	score	horas
101	Juan	5	10
102	Ana	6	8
103	Pedro	5	15

Tabla de Precios por Score:

score	valorHora
5	5000
6	6000

Al separar la tabla, eliminamos la redundancia y nos aseguramos de que cada tabla tenga una buena dependencia funcional (`gid` $\rightarrow$... y `score` $\rightarrow$...). Esto nos permite modificar, insertar o eliminar un `valorHora` sin afectar los datos de los guías.

Formas Normales

Las formas normales son un conjunto de reglas que aplicamos a una tabla para evaluar si su diseño es "bueno". Si una tabla no cumple con una forma normal, se puede **descomponer** en tablas más pequeñas que sí lo hagan, eliminando la redundancia.

- **Primera Forma Normal (1NF):** La regla más básica. Un esquema está en 1NF si todos sus atributos son **atómicos** (no se pueden dividir) y sus valores son únicos para cada celda. Esto significa que no hay grupos repetidos, y la información está organizada en filas y columnas.
Ejemplo (Violación de 1NF): Considera la tabla **CLIENTES** que vimos en la clase 1, pero con una columna para múltiples números de teléfono:

ID_Cliente	Nombre	Saldo	Teléfonos
C001	Juan Pérez	\$5,000,000	111-222, 333-444
C002	María López	\$12,500,000	555-666

La columna **Teléfonos** no es atómica, ya que contiene más de un valor en la primera fila.

ID_Cliente	Nombre	Saldo
C001	Juan Pérez	\$5,000,000
C002	María López	\$12,500,000

Nueva Tabla de Teléfonos:

ID_Cliente	Número de Teléfono
C001	111-222
C001	333-444
C002	555-666

- **Segunda Forma Normal (2NF):** Una tabla está en 2NF si está en 1NF y todos los atributos que no son parte de la llave primaria dependen completamente de ella. Dicho de otra forma, esta forma normal se centra en las tablas que tienen llaves primarias compuestas (formadas por dos o más atributos). En estas tablas, ningún atributo no-llave debe depender solo de una parte de la llave, sino de la llave completa.
Ejemplo (Violación de 2NF): Imagina una tabla que registra a qué empleado se le asigna cada cuenta del banco, con una llave primaria compuesta por **ID_Empleado** y **Nro_Cuenta**. La tabla también incluye el nombre del empleado.

ID_Empleado	Nro_Cuenta	Nombre_Empleado
E001	1001	Carlos
E001	1002	Carlos
E002	1003	Ana

El atributo **Nombre_Empleado** no depende de toda la llave (**ID_Empleado** , **Nro_Cuenta**), sino solo de una parte de ella (**ID_Empleado**). Esto es una **dependencia parcial**.

Solución (Convertir a 2NF): Separamos la información del empleado en una tabla distinta.

Tabla de Asignaciones:

ID_Empleado	Nro_Cuenta
E001	1001
E001	1002
E002	1003

Tabla de Empleados:

ID_Empleado	Nombre_Empleado
E001	Carlos
E002	Ana

- **Tercera Forma Normal (3NF):** Una tabla está en 3NF si está en 2NF y no tiene dependencias transitivas. Una dependencia transitiva ocurre cuando un atributo no-llave depende de otro atributo no-llave, en lugar de la llave primaria.
Ejemplo (Violación de 3NF): Volvamos al ejemplo de los guías turísticos.

gid	nombre	score	horas	valorHora
101	Juan	5	10	5000
102	Ana	6	8	6000
103	Pedro	5	15	5000

La llave primaria es **gid**. El atributo **valorHora** no depende directamente de **gid**, sino que depende del atributo **score**, que a su vez depende de **gid**. Esto es una **dependencia transitiva** (gid→score→valorHora).

Solución (Convertir a 3NF): Para normalizar, separamos la información de las dependencias transitivas en una nueva tabla.

Tabla de Guías:

gid	nombre	score	horas
101	Juan	5	10
102	Ana	6	8
103	Pedro	5	15

Tabla de Precios por Score:

score	valorHora
5	5000
6	6000

- **Forma Normal de Boyce-Codd (BCNF):** Una tabla está en BCNF si por cada dependencia funcional no trivial (X→Y), **X** es una **superllave**. Una superllave es un conjunto de atributos que identifica una tupla de forma única. BCNF es una forma más estricta de 3NF que resuelve algunas anomalías restantes.
 - En términos sencillos, BCNF se asegura de que todas las dependencias funcionales estén basadas en una llave de la tabla. Si una tabla no está en BCNF, es probable que la estés guardando con información redundante o que su diseño sea incompleto.

Clase 6 y 7: SQL - Structured Query Language

SQL (Structured Query Language) es el lenguaje estándar utilizado para comunicarnos con las bases de datos relacionales. A diferencia de un lenguaje de programación procedural, SQL es **declarativo**: le decimos al sistema *qué* datos queremos, pero no le decimos *cómo* debe encontrarlos. El Sistema de Gestión de Bases de Datos (SGBD) es el encargado de convertir nuestra consulta en un plan de ejecución eficiente. SQL se divide en dos categorías principales:

- **DDL (Data Definition Language):** Lenguaje de **Definición de Datos**. Se usa para crear, modificar y eliminar la estructura de las tablas, atributos y restricciones.
- **DML (Data Manipulation Language):** Lenguaje de **Manipulación de Datos**. Se usa para consultar, insertar, modificar y eliminar los datos (tuplas) que se encuentran dentro de las tablas.

Tipos de Datos Comunes

Para definir los atributos de una tabla, usamos tipos de datos que indican la naturaleza de la información:

- **Caracteres (Strings)**
 - **CHAR(n)** : Cadena de longitud fija.
 - **VARCHAR(n)** : Cadena de longitud variable.
- **Números**

- `INT` : Números enteros.
- `FLOAT` : Números con decimales.
- **Fechas y Tiempo**
 - `DATE` : Almacena solo la fecha.
 - `TIME` : Almacena solo la hora.
 - `TIMESTAMP` : Almacena fecha y hora.

DDL: Definición de la Estructura

- `CREATE TABLE` **(Crear Tabla)**

Esta es la instrucción principal para definir una nueva tabla, sus atributos y las llaves. **Sintaxis General:**

```
CREATE TABLE <nombre_tabla> (  
  <atributo_1> <tipo_dato> [opciones],  
  <atributo_2> <tipo_dato> [opciones],  
  ...  
  <restricciones_de_tabla>  
);
```

Opciones y Restricciones:

- `PRIMARY KEY` : Designa un atributo como llave primaria, asegurando que sus valores sean únicos y no nulos.
- `FOREIGN KEY` : Crea una llave foránea que referencia a la llave primaria de otra tabla, estableciendo una relación.
- `DEFAULT` : Asigna un valor por defecto si no se especifica uno al insertar una tupla.
- `NOT NULL` : Obliga a que el atributo tenga un valor y no pueda ser nulo.

Ejemplo de Creación de Tablas (usando el modelo del banco):

```
-- Creamos la tabla de Clientes  
CREATE TABLE Clientes (  
  ID_Cliente VARCHAR(10) PRIMARY KEY,  
  Nombre VARCHAR(50) NOT NULL,  
  Saldo FLOAT  
);  
  
-- Creamos la tabla de Cuentas, con una llave foránea a Clientes  
CREATE TABLE Cuentas (  
  Nro_Cuenta VARCHAR(10) PRIMARY KEY,  
  ID_Cliente VARCHAR(10) NOT NULL,  
  Tipo VARCHAR(20) DEFAULT 'Ahorro',  
  Saldo FLOAT,  
  FOREIGN KEY (ID_Cliente) REFERENCES Clientes(ID_Cliente)  
);
```

- `ALTER TABLE` **(Modificar Tabla)**

Se usa para modificar la estructura de una tabla existente sin eliminarla.

- **Agregar una columna:** `ALTER TABLE Peliculas ADD COLUMN productor VARCHAR(30);`
- **Eliminar una columna:** `ALTER TABLE Peliculas DROP COLUMN director;`

- `DROP TABLE` **(Eliminar Tabla)**

Borra una tabla completa de la base de datos.

```
DROP TABLE Peliculas;
```

DML: Manipulación de los Datos

- `INSERT INTO` **(Insertar Filas)**

Permite agregar nuevas tuplas a una tabla. Puedes especificar los atributos a insertar o, si vas a insertar todos, omitir la lista de atributos.

Ejemplo:

```
-- Sintaxis completa, especificando los atributos  
INSERT INTO Clientes (ID_Cliente, Nombre, Saldo)  
VALUES ('C004', 'Ana García', 8500000.00);  
  
-- Sintaxis abreviada, si los valores coinciden con el orden de las columnas  
INSERT INTO Clientes  
VALUES ('C005', 'Luis Fernández', 3200000.00);
```

- `SELECT` **(Consultar Datos)**

La consulta más común. Selecciona los datos de una o más tablas basándose en ciertas condiciones.

Sintaxis General:

```
SELECT <lista_de_atributos>
FROM <tablas>
WHERE <condiciones_opcional>
```

- `SELECT *` selecciona todos los atributos.
- El `WHERE` es opcional y se usa para filtrar las filas. Permite operadores de comparación (`=` , `!=` , `<` , `>`), lógicos (`AND` , `OR` , `NOT`) y especiales (`IN` , `BETWEEN`).

Ejemplos:

```
-- Para ver todos los datos de la tabla Clientes:
SELECT *
FROM Clientes;

-- Para ver el nombre y el saldo de los clientes con saldo mayor a $10,000,000:
SELECT Nombre, Saldo
FROM Clientes
WHERE Saldo > 10000000.00;

-- Clientes con ID entre 'C001' y 'C003':
SELECT *
FROM Clientes
WHERE ID_Cliente BETWEEN 'C001' AND 'C003';

-- Clientes con ID 'C001' o 'C005':
SELECT *
FROM Clientes
WHERE ID_Cliente IN ('C001', 'C005');
```

- **UPDATE (Actualizar Datos)**

Modifica los valores de una o más tuplas. Es crucial usar la cláusula `WHERE` para evitar actualizar toda la tabla.

Ejemplo:

```
-- Actualiza el saldo del cliente 'C001' a $6,500,000
UPDATE Clientes
SET Saldo = 6500000.00
WHERE ID_Cliente = 'C001';
```

- **DELETE (Eliminar Datos)**

Elimina una o más tuplas de una tabla. Al igual que `UPDATE` , es vital usar `WHERE` .

Ejemplo:

```
-- Elimina el cliente con ID 'C005'
DELETE FROM Clientes
WHERE ID_Cliente = 'C005';
```

Operaciones con Múltiples Tablas

- **JOIN (Unir Tablas)**

Combina filas de dos o más tablas en una sola. La forma más común es el `INNER JOIN` .

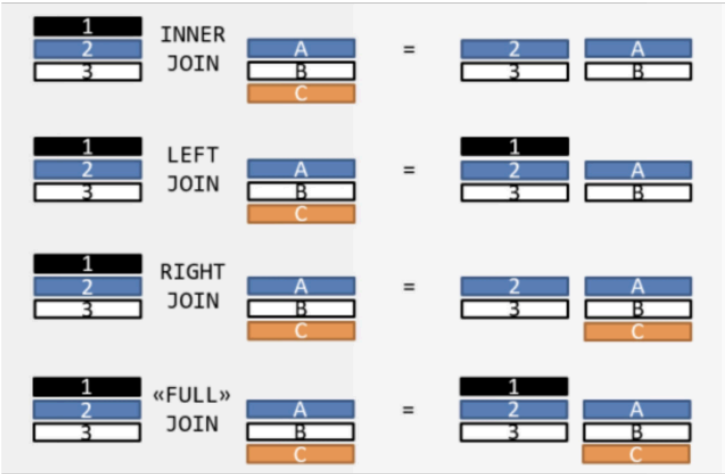
Ejemplo con `INNER JOIN` :

Para obtener el nombre del cliente y el tipo de cuenta de cada cuenta que posee, unimos las tablas `Clientes` y `Cuentas` donde el `ID_Cliente` coincida en ambas.

```
SELECT C.Nombre, T.Nro_Cuenta, T.Tipo
FROM Clientes AS C
INNER JOIN Cuentas AS T
ON C.ID_Cliente = T.ID_Cliente;
```

Tipos de `JOIN`

- `INNER JOIN` : Devuelve solo las filas que tienen una coincidencia en ambas tablas.
- `LEFT (OUTER) JOIN` : Devuelve todas las filas de la tabla izquierda, y las que coincidan de la derecha. Si no hay coincidencia, los valores de la tabla derecha serán `NULL` .
- `RIGHT (OUTER) JOIN` : Similar a `LEFT JOIN` , pero devuelve todas las filas de la tabla derecha.
- `FULL (OUTER) JOIN` : Devuelve todas las filas de ambas tablas, llenando con `NULL` donde no haya coincidencia.



- **GROUP BY** y **HAVING**
 - **GROUP BY** : Agrupa las filas que tienen los mismos valores en las columnas especificadas, para que las funciones de agregación se apliquen a cada grupo por separado. **Los atributos en el GROUP BY deben estar en el SELECT** .
 - **HAVING** : Similar a **WHERE** , pero filtra los **grupos** después de la agregación. No se puede usar **WHERE** con funciones de agregación.

Ejemplo:

```
-- Calcula el saldo promedio por cliente, pero solo para aquellos que tienen un saldo total superior a $8,000,000
SELECT ID_Cliente, SUM(Saldo) AS SaldoTotal
FROM Cuentas
GROUP BY ID_Cliente
HAVING SUM(Saldo) > 8000000.00;
```

Funciones de Agregación

Las funciones de agregación operan sobre un conjunto de valores para devolver un único valor resumido.

- **AVG()** : Promedio.
- **COUNT()** : Cuenta las filas. **COUNT(*)** cuenta todas, **COUNT(atributo)** cuenta las filas no nulas.
- **SUM()** : Suma.
- **MIN()** : Valor mínimo.
- **MAX()** : Valor máximo.

Ejemplo:

```
-- Cuenta el número total de clientes
SELECT COUNT(*) FROM Clientes;
```

Consultas Anidadas y Operadores de Conjunto

Consultas Anidadas

Una consulta anidada es una **SELECT** dentro de otra **SELECT** .

Ejemplo:

```
-- Obtiene el nombre del cliente que tiene el saldo más alto
SELECT Nombre
FROM Clientes
WHERE Saldo = (SELECT MAX(Saldo) FROM Clientes);
```

Operadores de Conjunto

Permiten combinar los resultados de dos o más consultas.

- **UNION** : Combina los resultados y elimina duplicados.
- **UNION ALL** : Combina los resultados y mantiene duplicados.
- **INTERSECT** : Devuelve solo las filas que aparecen en ambas consultas.
- **EXCEPT** : Devuelve las filas de la primera consulta que no están en la segunda.

Manejo de Valores Nulos (**NULL**)

Un valor **NULL** representa un dato desconocido o no aplicable. Los valores nulos no son iguales a cero o a una cadena vacía.

- **IS NULL** y **IS NOT NULL** : Se usan para verificar si un atributo tiene un valor nulo.

```
-- Selecciona los clientes que no tienen un saldo registrado
SELECT *
FROM Clientes
WHERE Saldo IS NULL;
```

- **NOT NULL** : Una restricción que se define al crear la tabla para prohibir valores nulos.